Sentinel x Apertus - Track 2B technical report

Track: 2B - Own Project. Event: Hack Apertus Online 2026.
Project/team label: Sentinel x Apertus. Contributor shown in the current workspace: Alen Sirola; final team membership declaration remains with the submitter.
Demo: PUBLIC_VIDEO_URL_PENDING. Model: swiss-ai/Apertus-v1.5-8B through the CSCS OpenAI-compatible endpoint.

Purpose and architecture

This prototype tests process continuity when real Apertus proposals are separated from human authority, execution and readback. The shared GenericTransition has transition_id, subject, observed_state, proposed_action, authority_required, execution_adapter, expected_effect and status. A status-independent digest locks its contents. The process layer controls ordered status changes and gate-issued execution permits. Domain mappings define action type, required authority, execution carrier and expected effect.

Irrigation is an isolated filesystem benchmark, not a physical actuator. A real Apertus proposal for A1 is mapped to the existing irrigation contract. The operator's explicit invocation authorizes the fixture. Worker A persists DECIDED and exits; the original real Motor B loads that state, confirms A1 through CSCS, and the unchanged CORE executes and verifies FIELD on/version 1. The generic layer records EXECUTED and performs a fresh readback before VERIFIED. CORE and generic digests are separate; each stays stable within its own handoff/transition.

For `Rumenka je betežna.`, real Apertus Motor A emits four constrained sections: observed user report, relevant known meaning, explicit unknowns and a context-request candidate. Species, symptoms, onset, severity, cause and authority remain unknown. Closed vocabulary rejects diagnosis, treatment, tool calls, invalid JSON and invented knowledge. The same GenericTransition receives the proposal, but has no execution adapter or expected effect. It stops above CORE at AWAITING_AUTHORITY. The scene is not executed or verified. Its legitimate stop is part of the proof acceptance.

Reproduction and validation

Python 3.12+ standard library, GNU Make, a CSCS credential and internet access are required. Run make test and make run with the secret supplied only through the runtime environment. The executable has no dependency on old receipts, private folders, an AWM book or third-party Python packages. Fresh real proposals and raw provider envelopes are retained for each run, with source hashes for provenance.

The final clean-copy test is documented by an external validation report and repo-relative receipts. Acceptance requires irrigation VERIFIED/K CLOSED, fresh real Apertus responses, ordered generic snapshots, stable locks, unchanged CORE, and Rumenka AWAITING_AUTHORITY with no execution or readback. The separate original real-provider regression must pass too. No mock fallback exists in either real path.

Twenty tests cover the original boundary cases and generic ordering, unknown adapters, absent/mismatched authority, gate bypass, invalid transitions, locked content mutation, failed readback, and repeat execution. Unit-test doubles are confined to tests and cannot be selected by the real runner. Docker build/runtime remain UNVERIFIED on the validation host where Docker is unavailable.

Provenance, value and limitations

Sentinel CORE is pre-existing and byte-identical, SHA256 3394a5b975998c802486eff3d61a8877346cb669f417bd0284fa38d0c726fce9. CSCS adapters, constrained interpreter, GenericTransition, mapping/orchestration, tests and packaging are hack-new. Source chronology and eligibility of reuse must be verified separately; this report is not an eligibility certificate.

Purposeful AI is demonstrated by bounded interpretation with retained uncertainty and a real execution-confirmation path. This does not measure improved reasoning, clinical usefulness or production robustness. Sovereign deployment has a local Python/container orchestration path using the CSCS inference endpoint; offline model hosting is not implemented and provider data-policy compliance is not proved. Cost and scalability remain unbenchmarked beyond the small prototype runs.

The caller, adapter code, registry and filesystem are trusted. The gate is not a sandbox or a cryptographic identity system. Crash recovery, concurrent execution, transactions and rollback after external effects are outside scope. There is only one actual execution adapter. A second authorized execution carrier would be needed before discussing generalization of CORE; the hack architecture is frozen.

Submission packaging and target architecture

The official repository keeps track_2b/src, data, docs, Makefile and Dockerfile. Repository-root make run now builds and launches Docker. LLM_API_KEY, LLM_NAME and LLM_BASE_URL are mapped by a thin container entrypoint to the existing CSCS variables; runtime and CORE are unchanged. No secret is saved in the image or tree. This new Docker entrypoint is statically checked but Docker remains UNVERIFIED; the previous native make run PASS is not presented as Docker PASS.

Target: on-premise process/container operated by its administrator, with CSCS inference as the external Swiss-hosted dependency. No air-gapped inference is implemented. Build-time dependency is python:3.12-slim from a container registry; runtime dependency is HTTPS access and a valid external CSCS credential. Full sovereign Swiss-cloud deployment and residency of the whole application are not measured on this local test host.

Data, evaluation and production tools

All fixture inputs are synthetic; Rumenka is a fictional text scene, not a patient record. No external dataset or human-subject data is redistributed. Fresh provider responses are generated into ignored local receipts. Acceptance metrics are exact gate/order/digest/readback assertions, not clinical accuracy. Existing baseline tests and real Motor B regression pass; no quantitative reasoning/cost/latency improvement is claimed. Tests use isolated doubles only; real runs cannot select a mock fallback.

The 110-second demo has embedded local Piper Lessac narration. Whisper base was used offline only as a production intelligibility check, not a runtime provider or clinical evaluator. Hero illustrations do not supply runtime model input. Public redistribution rights for the Lessac synthesized audio remain unconfirmed; weights/toolchains are not submitted. The Croatian phrase remains visible and runtime understanding is proved by real Apertus text inference.

Next steps and licensing

Hack architecture is frozen. Next steps are Docker execution on an available host, verified account/public artifact links and truthful ownership/eligibility declarations. Further real-world operation requires a real authority and execution carrier; this submission does not pretend that such a carrier exists.

Source/build files: Apache-2.0. Project documentation: CC-BY-4.0. Synthetic fixtures: CDLA-Permissive-2.0. Pre-existing CORE provenance and rights must be confirmed by its owner. No private background material is included.

References

https://github.com/HackApertus/project-template/tree/main/track_2b
https://hackapertus.ch/terms-and-conditions
https://docs.cscs.ch/services/inference/api/
https://huggingface.co/rhasspy/piper-voices/tree/main/en/en_US/lessac/medium